

Blockchain Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly. Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:

Contract 1: VotingSystem

Voting System :

```
// SPDX-License-Identifier: MIT pragma
```

```
solidity ^0.8.0;
```

```
contract VotingSystem {
```

```
    struct Candidate
    {
        uint id;
        string name;
        uint voteCount;
    }
```

```
    mapping(uint => Candidate) public candidates;
    mapping(address => bool) public voters;
```

```
    uint public candidatesCount;
```

```

function addCandidate(string memory _name) public {
    candidatesCount++;    candidates[candidatesCount] =
Candidate(candidatesCount, _name, 0);
}

```

```

function vote(uint _candidateId) public {    require(!voters[msg.sender], "Already
voted!");    require(_candidateId > 0 && _candidateId <= candidatesCount, "Invalid
candidate");

```

```

    voters[msg.sender] = true;
    candidates[_candidateId].voteCount++;
}

```

```

function getVotes(uint _candidateId) public view returns (uint) {
return candidates[_candidateId].voteCount;
}
}

```

Contract 2:

```

Voting Admin;
// SPDX-License-Identifier: MIT pragma
solidity ^0.8.0;

```

```

contract VotingAdmin {

```

```

    address public admin;
    bool public votingActive;

```

```

    constructor() {
admin = msg.sender;
    votingActive = false;
}

```

```

    modifier onlyAdmin() {
require(msg.sender == admin, "Not admin");
    _;
}

```

```

    function startVoting() public onlyAdmin {
    votingActive = true;
}

```

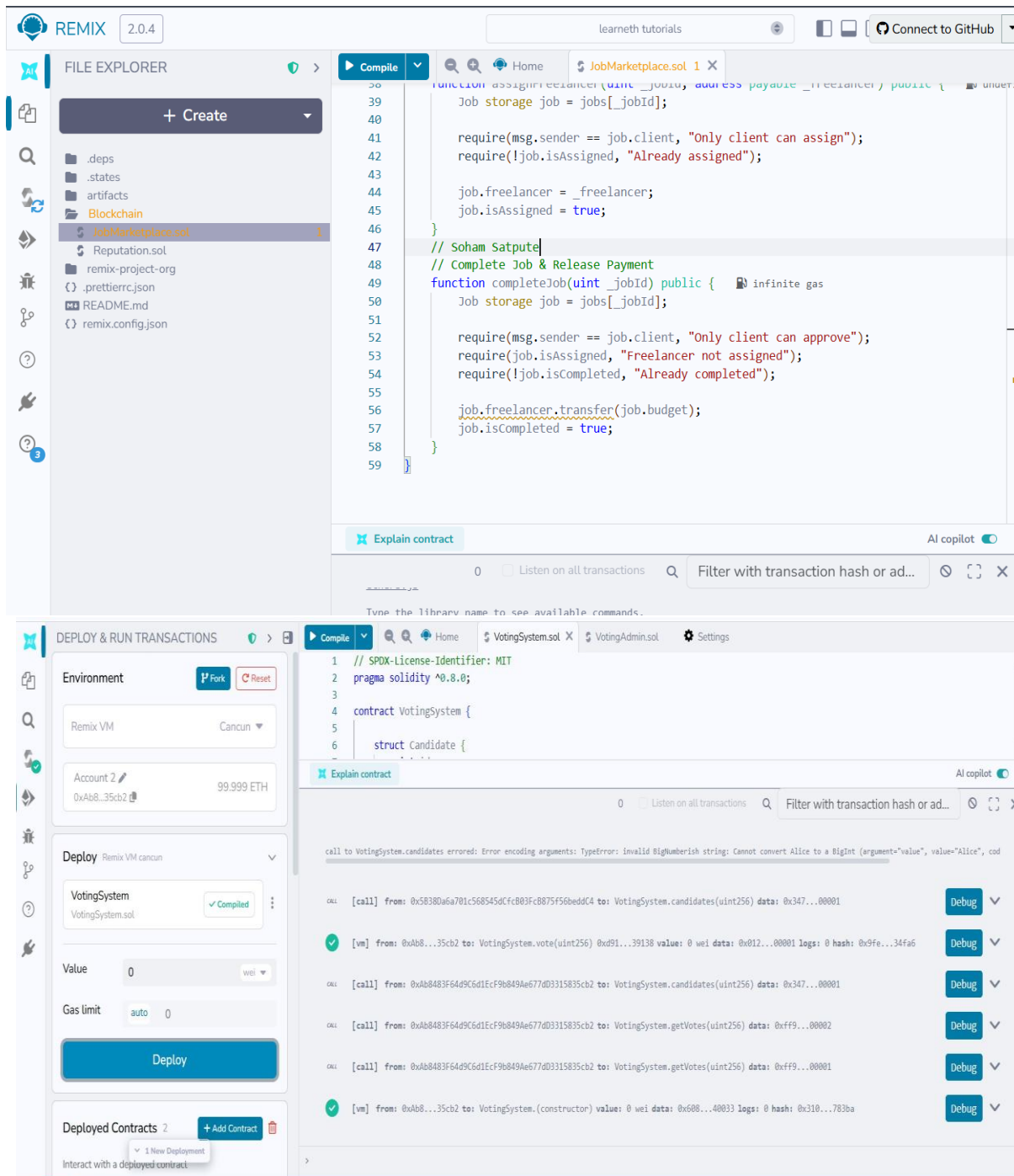
```

    function endVoting() public onlyAdmin {
    votingActive = false; }    function
getStatus() public view returns (bool) {
    return votingActive;
}
}

```

Implementation :

ScreenShots :



Transaction deployed successfully

Low level interaction

non-payable

addCandidate

string

"Bob"

Value

0

wei

Gas limit

auto

0

Transact

Low level interaction

non-payable

addCandidate

string

"Alice"

Value

0

wei

Gas limit

auto

0

Transact

Both Candidates Added

call

candidatesCount

0:uint256: 2

Call

Balance

0 ETH

Candidate Count

call candidates uint256

1

0:uint256: id 1
1:string: name Alice
2:uint256: voteCount 2

Call

call candidates uint256

2

0:uint256: id 2
1:string: name Bob
2:uint256: voteCount 0

Call

Candidates

call getVotes uint256

1

Value 0 wei
Gas limit auto 0

Transact

✓ [vm] from: 0x5B3...eddC4 to: VotingSystem.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00001 logs: 0 hash: 0x8f7...c9c5d

status

1 Transaction mined and execution completed

transaction hash

0x8f7ef33e383b9e55f8de9c0186ffaad0c8a25f5ae3cc3826979fd5ad47dc9c5d

block hash

0xb5764a0a768a4ddc85d5623eae0013740f6f1c4fc0eb5e90a8bcf443ba87683f

block number

6

from

0x5B38D6a7801c568545dCfcB03FcB875f56beddC4

to

VotingSystem.vote(uint256) 0xd9145CCE520386f254917e481e844e9943f39138

transaction cost

68570 gas

execution cost

47366 gas

output

0x

decoded input

{
 "uint256_candidateId": "1"
}

Vote

call getVotes uint256

1

0:uint256: 2

Call

Balance 0 ETH

call getVotes uint256

2

0:uint256: 0

Call

Balance 0 ETH

Vote Count

Conclusion:

The Blockchain-based Voting System was successfully implemented using Solidity smart contracts and deployed on the Remix IDE. Two smart contracts were designed to handle voting operations and administrative control, ensuring modularity and better system design.

The system allowed adding candidates, casting votes, and retrieving vote counts while preventing double voting through the use of mappings and conditions. All transactions were executed on the blockchain, ensuring transparency, security, and immutability of data.

Thus, the experiment helped in understanding the working of smart contracts, their deployment, and execution in a decentralized environment. The objectives of designing and implementing smart contracts were successfully achieved.